Aula 10 — Comunicação serial

O periférico que gera o protocolo sozinho

Microcontroladores — DENE/UFMT

OBJETIVOS

- Comparar a transmissão serial com a paralela do encontro 3 em pinos, sincronismo, distância e velocidade, e explicar por que as interfaces modernas são seriais.
- Distinguir comunicação síncrona de assíncrona pelo modo como o receptor sabe quando ler cada bit.
- Relacionar os protocolos seriais mais comuns (UART, RS-485, SPI, I²C, 1-Wire, CAN, USB) às aplicações em que cada um aparece.
- Distinguir um protocolo gerado por software de um gerado por periférico dedicado, e quantificar a diferença em ciclos bloqueados.
- Descrever o quadro assíncrono e explicar por que transmissor e receptor precisam concordar previamente com a taxa.
- Derivar o valor do divisor de taxa a partir de F_{osc}, calcular o erro resultante e julgá-lo contra o orçamento de erro do quadro.
- Diagnosticar uma ligação serial pela aparência do defeito, distinguindo tela em branco de texto corrompido.
- Implementar transmissão por interrupção com fila circular, e justificar por que a versão bloqueante é pior que o display.
- Decidir quando usar uma biblioteca pronta, sabendo quais escolhas ela toma no seu lugar.

1 O display mostra o instante

No encontro 9 ficaram duas grandezas para medir: a amplitude da oscilação e o número de acionamentos por hora. Nenhuma delas cabe numa tela de 16 colunas. Para contar acionamentos é preciso observar o sistema durante uma hora, e para ver a amplitude é preciso guardar os valores ao longo do tempo. O display só mostra o valor atual.

CONCEITO

A partir desta aula o termostato passa a mandar dados para um segundo computador, que os guarda. A questão da aula é como fazer essa comunicação sem atrapalhar o resto do programa.

2 Paralelo e serial

No encontro 3 o display recebeu os dados por um **barramento paralelo**: quatro (ou oito) fios de dado, um fio de endereço (`RS`) e um de habilitação (`E`). Os bits de um mesmo valor viajam lado a lado, cada um no seu fio, e a borda de `E` diz ao display o instante em que todos estão válidos.

Na transmissão **serial** os bits viajam um atrás do outro, pelo mesmo fio.

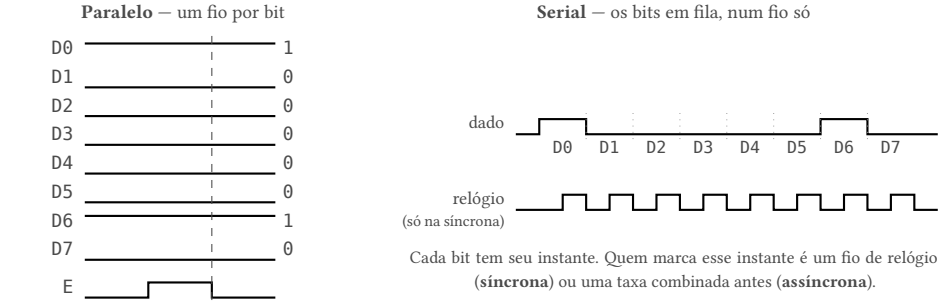


Figura 1: O byte `'A'` (0x41) pelos dois caminhos. No paralelo, oito fios e uma borda; no serial, um fio e oito instantes, um por bit.

	Paralelo (display, encontro 3)	Serial
Bits por vez	4 ou 8	1
Pinos	6 a 10	1 a 4
Quando ler	A borda de `E`	Um fio de relógio, ou uma taxa combinada antes
Distância	Centímetros	De centímetros a quilômetros
O que limita a velocidade	A diferença de atraso entre os fios	Só o próprio fio

CONCEITO

Paralelo não é mais rápido. A intuição diz que oito fios levam oito vezes mais bits que um, e isso vale enquanto o relógio é lento. Quando ele acelera, um detalhe passa a dominar: os fios nunca têm exatamente o mesmo comprimento e a mesma carga, e os bits de um mesmo byte chegam em instantes ligeiramente diferentes. O receptor precisa esperar o mais atrasado antes de ler, e essa espera cresce com a distância. Os fios vizinhos ainda interferem uns nos outros.

Um fio só não tem esse problema. Por isso a taxa de uma ligação serial pode subir até ultrapassar a paralela, e foi o que aconteceu: a porta de impressora deu lugar ao USB, os discos IDE ao SATA, o barramento PCI ao PCI Express. Todos trocaram dezenas de fios por um ou dois pares seriais, mais rápidos.

No microcontrolador pesa ainda mais o outro lado da tabela: pinos. O encontro 3 já mostrou que quatro pinos economizados decidem se o termostato cabe no kit.

2.1 Síncrona ou assíncrona

CONCEITO

No paralelo, o fio `E` marca o instante de ler. No serial a pergunta continua existindo, agora para cada bit, e há duas respostas.

Síncrona: um fio a mais leva o relógio, e cada borda dele marca um bit. O receptor não precisa saber a taxa de antemão: lê quando a borda chega. SPI e I²C funcionam assim (encontro 11).

Assíncrona: não há fio de relógio. Os dois lados combinam antes quanto tempo dura cada bit, e o receptor se sincroniza numa borda especial no começo de cada byte. A UART, desta aula, funciona assim.

A assíncrona economiza um fio e permite distâncias maiores, mas obriga os dois relógios a concordarem. A seção sobre o divisor de taxa é o preço dessa economia.

3 Muitos protocolos, uma aplicação cada

"Serial" não é um protocolo, é uma família. Os membros diferem no número de fios, no sincronismo, em quantos dispositivos dividem a linha e na distância que alcançam, e cada combinação se firmou numa aplicação.

Protocolo	Fios	Relógio	Onde aparece
UART (TTL ou RS-232)	2: TX e RX	Assíncrono	Ponto a ponto. Console e telemetria para o computador, módulos GPS, Bluetooth e GSM, bootloaders
RS-485	2: par diferencial	Assíncrono	Vários dispositivos na mesma linha, até cerca de 1 km. Automação industrial (Modbus)
SPI	3 + 1 seleção por escravo	Síncrono	Dentro da placa, a dezenas de MHz. Cartão SD, displays gráficos, conversores e memórias Flash externas

I²C	2: SDA e SCL	Síncrono	Dentro da placa, 100 ou 400 kHz, dispositivos endereçados. Sensores, EEPROM, relógio de tempo real
1-Wire	1 + terra	Assíncrono	Vários sensores num único fio. Termômetro DS18B20, chaves de identificação
CAN	2: par diferencial	Assíncrono	Vários mestres, com prioridade entre mensagens e imune a ruído. Automóveis, máquinas agrícolas (ISOBUS)
USB	2: par diferencial	Embutido no sinal	O computador é o mestre e enumera os dispositivos. Periféricos de PC, o bootloader do kit

CONCEITO

Quase nunca se escolhe o protocolo em abstrato. Quem escolhe é a aplicação, por três perguntas:

Com quem o chip vai falar? Um sensor comprado já vem com I²C ou SPI; um computador entende USB e porta serial. O outro lado costuma decidir sozinho.

A que distância, e com quanto ruído? Dentro da placa, SPI e I²C. Em cabo de metros num galpão ou num trator, RS-485 ou CAN, com par diferencial.

Quantos dispositivos dividem os fios? Só dois, UART. Vários, SPI (com um fio de seleção para cada) ou I²C, RS-485, 1-Wire e CAN, que distinguem os dispositivos por endereço ou prioridade.

O PIC18F4550 traz periféricos para três desses: a EUSART (UART), o MSSP (SPI e I²C) e o módulo USB. Esta aula usa a UART porque a aplicação é mandar a telemetria para o computador. O encontro 11 usa o I²C porque a aplicação é uma memória na mesma placa.

4 Quem gera o protocolo

Serial ou paralelo diz quantos fios a ligação usa. O encontro 3 deixou outra pergunta, que pesa mais para o programa: quem produz as bordas. No display era o seu código; a serial é o primeiro caso do curso em que as bordas são produzidas por um periférico.

	Display HD44780	**EUSART**
Quem move as linhas	O seu código, uma borda por instrução	O periférico, sozinho
O que o programa faz	Escreve `LAT`, espera, escreve, espera	Escreve um byte em `TXREG` e vai embora
Custo por byte	Cerca de 160 ciclos **bloqueados**	Alguns ciclos, e o hardware leva 1,04 ms **sem o processador**
Se o programa demorar	Nada acontece — o display espera	Nada acontece — o byte já está a caminho

CONCEITO

A diferença está na terceira linha. No display, o tempo de transmissão é tempo do processador. Na serial, o byte leva cerca de 1 ms para sair pelo fio e, nesse intervalo, o processador continua executando o programa.

O Timer0 do encontro 5 já contava sem ajuda do programa. A EUSART faz o mesmo com um protocolo inteiro.

5 O quadro

A UART é assíncrona: usa dois fios, um para cada sentido, e nenhum deles leva relógio. Por isso os dois lados precisam combinar a taxa antes de começar.

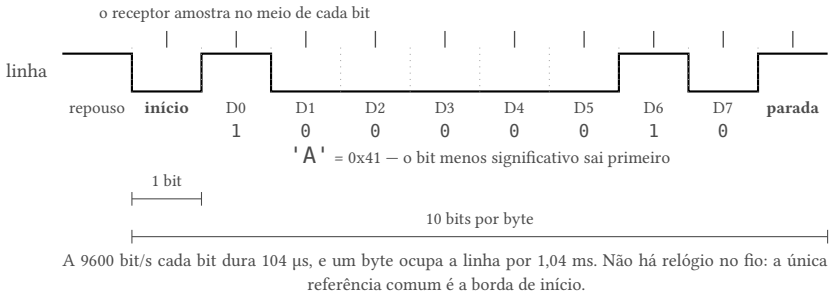


Figura 2: Quadro de dez bits. O receptor usa a borda de descida do bit de início como referência e, a partir dela, mede o tempo com o próprio relógio.

Em repouso a linha fica em nível alto. O bit de início, em nível baixo, cria a borda de descida que o receptor detecta. CONCEITO

O receptor espera meio tempo de bit para chegar ao centro do bit de início e, a partir daí, amostra a linha a cada tempo de bit. Os oito bits de dado saem do menos significativo para o mais significativo. O bit de parada, em nível alto, devolve a linha ao repouso.

São dez tempos de bit para oito bits de dado, ou 80% de eficiência. Os dois bits a mais são o custo de não ter um fio de relógio.

5.1 O divisor, e por que a taxa quase nunca é exata

A taxa não é programada diretamente em bits por segundo. O que se programa é um divisor inteiro do oscilador, e a taxa resulta dele.

Com o gerador de 16 bits e `BRGH` = 1, a relação é CONCEITO

$$\text{taxa} = F_{\text{osc}} / (4(n+1))$$

Para 9600 bit/s a 16 MHz:

$$n = 16 \cdot 10^6 / (4 \cdot 9600) - 1 = 415,67$$

Como o divisor precisa ser inteiro, arredondamos para $n = 416$:

$$\text{taxa real} = 16 \cdot 10^6 / (4 \cdot 417) = 9592,3 \text{ bit/s}$$

$$\text{erro} = -0,08\%$$

Taxa desejada	n	Taxa real	Erro	Veredito
9600	416	9592,3	− 0,08%	Ótimo
9600 (8 bits, `BRGH`)	103	9615,4	+ 0,16%	Também serve
115200	34	114 285,7	− 0,79%	Serve, com menos margem
115200 ($n = 33$)	33	117 647	+ 2,1%	Arriscado

Qual erro é aceitável. O receptor amostra o bit de parada 9,5 tempos de bit depois da borda de início. Se até lá o desvio acumulado passar de meio tempo de bit, a amostra cai no bit vizinho: CONCEITO

$$0,5 / 9,5 \approx 5,3\%$$

Esse limite vale para o enlace, ou seja, para a soma dos erros dos dois lados. Como o computador também tem o seu erro, a regra prática é manter cada lado abaixo de 2%.

Por isso os 2,1% da última linha da tabela são arriscados: sozinhos eles cabem no limite, mas sobra pouca margem para o outro lado.

5.2 Diagnóstico pela aparência do defeito

CONCEITO

Dois defeitos comuns têm sintomas diferentes, e dá para distinguir um do outro sem osciloscópio.

Nada aparece no terminal. O programa do computador não aceitou a taxa. Isso acontece quando o divisor calculado produz uma taxa fora da lista padrão: 9592 é aceito como 9600, mas 7350 não corresponde a nenhuma. O driver não mostra mensagem de erro, só não abre a porta.

Aparece texto corrompido, com um padrão que se repete. Os dois lados aceitaram a configuração, mas com taxas diferentes. Os quadros são lidos com amostras nos pontos errados e, como o erro é sempre o mesmo, o lixo também se repete.

6 Níveis e o que é USB

O pino do microcontrolador vai de 0 a 5 V. O RS-232 usa níveis invertidos, da ordem de ± 12 V, e o USB nem é uma porta serial.

NESTE KIT

A placa tem um conversor entre os níveis do pino e os que o computador espera. Sem ele, ligar o pino direto num conector DE-9 de RS-232 aplicaria − 12 V num pino que suporta − 0,3 V. O que os diodos de proteção fazem nesse caso foi visto no encontro 8.

Verificar antes do R10: qual conversor a placa usa, e em quais pinos o RX e o TX do microcontrolador estão ligados.

6.1 Vinte minutos sobre USB

CONCEITO

O bootloader que vocês usam toda semana funciona por USB, e até agora não falamos dele.

O USB é um barramento em que o computador é o mestre: ele endereça os dispositivos, organiza o tráfego em pacotes com verificação de erro e faz a **enumeração**. Quando um dispositivo é conectado, ele envia uma descrição de si mesmo, e o computador escolhe o driver a partir dela.

A porta COM virtual que aparece no computador é uma emulação. Existe uma classe de dispositivo USB que se declara como porta serial, e o sistema operacional cria a porta para ela. Nesse caminho não há UART.

O bootloader do kit usa outra classe, a de dispositivos de interface humana (HID), a mesma de teclados e mouses. Todo sistema operacional já traz driver para essa classe, e por isso o bootloader funciona sem instalar nada.

NOTA

Isso explica um número da tabela de frequências do encontro 2, que dizia que 48 MHz é o que o módulo USB exige, e não o que o núcleo usa. A temporização do USB é fixada pela especificação, e o módulo precisa de um relógio que a produza exatamente. O núcleo pode rodar a 16 MHz porque não depende disso.

7 Ligar o periférico

```
void uart_iniciar(void)
{
    TRISCbits.TRISC6 = 0;      /* TX: saida */
    TRISCbits.TRISC7 = 1;      /* RX: entrada */

    BAUDCONbits.BRG16 = 1;     /* gerador de 16 bits           */
    TXSTAbits.BRGH    = 1;     /* alta velocidade -> divisor 4 */
    SPBRGH = 0x01;             /* 416 = 0x01A0                 */
    SPBRG  = 0xA0;
```

```
    TXSTAbits.SYNC = 0;        /* assincrono */
    RCSTAbits.SPEN = 1;        /* liga o periferico, assume os dois pinos */
    TXSTAbits.TXEN = 1;        /* habilita transmissao */
    RCSTAbits.CREN = 1;        /* habilita recepcao    */
}
```

NOTA

Neste bloco, só `SPBRGH:SPBRG` exigiu conta: é o 416 da seção anterior. O resto são escolhas de configuração: assíncrono ou síncrono, divisor de 8 ou 16 bits, alta ou baixa velocidade.

Atenção ao `SPEN`. Quando ele é ligado, os dois pinos passam a ser controlados pelo periférico, e escrever em `LATC6` deixa de ter efeito no pino. Até aqui, escrever no `LAT` sempre mudava o pino; esta é a primeira exceção do curso.

8 Enviar sem bloquear

A forma mais simples de enviar é esperar o transmissor ficar livre e escrever o byte. Também é a mais cara para o resto do programa.

```
void uart_byte(uint8_t b)
{
    while (!PIR1bits.TXIF) ;    /* espera a fila esvaziar */
    TXREG = b;
}
```

A 9600 bit/s cada byte ocupa a linha por 1,04 ms, então uma linha de telemetria com 25 caracteres bloqueia o processador por 26 ms.

A atualização de tela do encontro 3 bloqueava por 1,3 ms e já era a operação mais lenta do sistema. A transmissão bloqueante é vinte vezes mais lenta, e passa do meio período de 1,14 ms da nota do buzzer e do intervalo de 5 ms do filtro de repique.

Mesmo a 115200 bit/s a linha leva 2,17 ms, mais que o display.

CONCEITO

A solução é a mesma do encontro 7. O programa coloca a linha numa fila circular e segue em frente. Quando o transmissor fica livre, ele pede uma interrupção, e o tratamento tira um byte da fila e o escreve em `TXREG`.

Cada byte custa algumas dezenas de ciclos, distribuídos ao longo do tempo. Com uma linha por segundo, isso dá menos de 0,1% do processador, e o programa principal nunca fica esperando.

Praticamente todo sistema embarcado que transmite dados usa esse esquema.

O código tem três partes: a fila, que o programa principal enche; o tratamento de interrupção, que a esvazia; e as duas linhas que ligam as interrupções.

```
#define FILA 64
static volatile uint8_t fila[FILA];
static volatile uint8_t entra = 0;     /* so o main escreve */
static volatile uint8_t sai = 0;       /* so a ISR escreve  */

/* Chamada so pelo main, nunca pela ISR. */
void uart_enfileirar(uint8_t b)
{
    uint8_t prox = (uint8_t)((entra + 1u) % FILA);
    while (prox == sai) { }            /* fila cheia: raro, e aqui sim espera */
    fila[entra] = b;
    entra = prox;
    PIE1bits.TXIE = 1;                 /* acorda o transmissor */
```

```
}

void __interrupt() isr(void)
{
    if (PIE1bits.TXIE && PIR1bits.TXIF) {     /* TXIF = 1: TXREG livre */
        if (entra == sai) {
            PIE1bits.TXIE = 0;          /* nada a enviar: desliga a fonte */
        } else {
            TXREG = fila[sai];          /* escrever em TXREG limpa TXIF */
            sai = (uint8_t)((sai + 1u) % FILA);
        }
    }
    /* a recepcao (secao "Receber") entra aqui, como um segundo if */
}

/* em main(), depois de uart_iniciar(): */
INTCONbits.PEIE = 1;                    /* a EUSART e periferica */
INTCONbits.GIE  = 1;
```

ATENÇÃO

Sem `PEIE` e `GIE` o programa compila e não dá erro, mas nada sai pelo fio. O pedido de interrupção nunca é atendido, os primeiros 63 bytes enchem a fila, e o 64.º trava o programa no `while` da fila cheia. Visto de fora, o sintoma é o mesmo de um cabo solto.

NOTA

Três escolhas deste código precisam de explicação.

`TXIE` é ligado ao enfileirar e desligado quando a fila esvazia. Se ficasse sempre ligado, o transmissor livre pediria interrupção o tempo todo, mesmo sem nada para enviar, e o programa principal deixaria de rodar (Exercício 10.3).

A espera com a fila cheia é a única que restou. Ela só acontece se o programa gerar dados mais rápido do que a serial consegue enviar. Se isso for frequente, aumentar a fila não resolve: é preciso enviar menos ou usar uma taxa maior.

Por causa dessa espera, `uart_enfileirar` (e o `printf`, que a usa) só pode ser chamada pelo programa principal. Dentro do tratamento, com a fila cheia, ela esperaria pelo próprio tratamento, que não roda enquanto ela espera: o programa trava.

Cada índice tem um único dono: só o `main` escreve `entra`, e só o tratamento escreve `sai`. Como os dois têm um byte, cada escrita é uma instrução só e não pode ser interrompida no meio. Por isso a fila dispensa desligar as interrupções.

9 Formatar sem gastar a Flash

Para montar a linha `125300,398,400,1,17` é preciso converter números em texto. O jeito mais curto é o `printf`, mas ele ocupa bastante memória.

```
/* Redireciona printf para a fila da serial. Basta definir putch. */
void putch(char c)
{
    uart_enfileirar((uint8_t) c);
}

/* ... e a partir daqui: */
printf("%lu,%d,%d,%u,%u\r\n", ms, temp_d, alvo_d, aquecedor, comutacoes);
```

ATENÇÃO

O `printf` ocupa da ordem de 1 a 2 kB de Flash, entre 3% e 6% dos 32 kB do chip, só para o formatador. Com ponto flutuante ocuparia muito mais, e a versão gratuita do compilador não oferece essa opção (encontro 4).

O tamanho exato aparece no arquivo de mapa gerado na compilação. Compare o mapa antes e depois de incluir o `printf`, em vez de confiar nesta estimativa.

Uma função própria ocupa algumas dezenas de bytes: CONCEITO

```
/* Escreve um inteiro sem sinal na fila, sem printf. */
static void enviar_u16(uint16_t v)
{
    char buf[6];
    uint8_t i = 0;

    do {
        buf[i++] = (char)('0' + (v % 10u));
        v /= 10u;
    } while (v != 0u);

    while (i != 0u) {
        uart_enfileirar((uint8_t) buf[--i]);   /* na ordem inversa */
    }
}
```

Na prática: use o `printf` enquanto desenvolve, confira o mapa e troque por funções próprias só se faltar espaço. Como no encontro 3, primeiro se mede, depois se otimiza.

10 Receber

A transmissão resolve a telemetria. A recepção permite o caminho contrário, por exemplo ajustar o alvo pelo computador em vez de usar o botão.

A recepção também pede interrupção, mas só se for habilitada. Em `uart_iniciar`, depois de `CREN`:

```
    PIE1bits.RCIE = 1;            /* byte recebido pede interrupcao */
```

No tratamento, a recepção é um segundo `if`, depois do da transmissão:

```
static volatile char linha[24];
static volatile uint8_t n = 0;
static volatile uint8_t linha_pronta = 0;

    /* dentro de isr(), depois do if da transmissao */
    if (PIE1bits.RCIE && PIR1bits.RCIF) {
        if (RCSTAbits.OERR) {          /* estouro: a recepcao PAROU */
            RCSTAbits.CREN = 0;
            RCSTAbits.CREN = 1;        /* religar e a unica saida   */
        }

        char c = RCREG;                /* ler RCREG limpa RCIF */

        if (linha_pronta) {
            /* o main ainda nao leu a anterior: descarta */
        } else if (c == '\n' || c == '\r') {
            if (n > 0u) {              /* ignora o \n de um \r\n */
                linha[n] = '\0';
                linha_pronta = 1;
                n = 0;
            }
        } else if (n < sizeof(linha) - 1u) {
            linha[n++] = c;
        }
    }
```

```
/* no laco do main */
if (linha_pronta) {
    printf("recebido: %s\r\n", (const char *) linha);
    linha_pronta = 0;              /* so agora a ISR pode reusar o buffer */
}
```

NOTA

Os terminais costumam mandar `\r\n` no fim da linha. Sem o teste `n > 0`, o `\r` fecharia a linha e o `\n` fecharia em seguida outra, vazia, por cima da primeira, antes de o `main` ler. O teste de `linha_pronta` protege o mesmo buffer: enquanto o `main` não termina de usar a linha, os bytes que chegam são descartados em vez de reescrevê-la pela metade.

ATENÇÃO

O estouro de recepção é o defeito mais difícil de diagnosticar desta aula.

O receptor guarda até dois bytes enquanto o programa não os lê. Se chegar um terceiro antes disso, o indicador `OERR` é ligado e a recepção para. Não se perde só um byte: nenhum byte novo é recebido até que o programa desligue e religue `CREN`.

Na bancada, a serial funciona por alguns minutos e depois para de receber, sem que o programa trave ou reinicie. Como a transmissão continua normal, a suspeita costuma cair no cabo ou no computador.

Em geral a causa é um tratamento de interrupção demorado, por exemplo um que escreve no display.

NOTA

Ler `RCREG` é o que zera `RCIF`. Se o tratamento testa `RCIF` e não lê `RCREG`, ele volta a ser chamado logo em seguida, sem parar. É o mesmo problema do `TXIE` sempre ligado.

NOTA

O programa inteiro, com as peças desta aula juntas, está em `code/uart/main.c` no repositório da disciplina. Ele manda uma saudação, depois `t = N s` a cada segundo, e devolve cada linha digitada no terminal. Foi testado em simulação a 9600 bit/s: um byte sai a cada 4170 ciclos, 1,04 ms, que é o tempo de um quadro a 9592 bit/s.

11 E as bibliotecas?

Existem, e a esta altura do curso vale usá-las.

Opção	O que entrega	O que decide por você
Configurador gráfico da Microchip	`uart_iniciar`, escrita e leitura, com ou sem fila	O divisor, o tamanho da fila, e se a escrita bloqueia
Bibliotecas de periférico antigas do compilador	`OpenUSART`, `putsUSART`	O mesmo, e estão descontinuadas
`printf` com `putch`	Toda a formatação	1 a 2 kB de Flash
`Serial` do Arduino	Tudo, inclusive a fila	Tudo

CONCEITO

O curso não proíbe bibliotecas. A ideia é fazer a conta uma vez à mão e depois usar a biblioteca sabendo o que ela faz.

Quando o configurador mostra uma caixa com “9600”, ele programou 9592,3 bit/s, com erro de 0,08%, que cabe no orçamento. Quem já fez a conta sabe disso; quem não fez não tem como avaliar a escolha.

A biblioteca também decide, sem avisar, **se a escrita bloqueia ou usa fila**. O configurador oferece as duas opções numa caixa de seleção, e a diferença entre elas são os 26 ms vistos nesta aula.

NOTA

Antes de usar uma biblioteca sem ler o código dela, verifique se você sabe o que ela faz em três situações: quando a fila enche, quando chega um byte e o anterior ainda não foi lido, e quando a função de escrita é chamada com o transmissor ocupado.

12 O que enviar

O formato importa para o encontro 13, em que esses dados vão servir de conjunto de treino em Python.

	Texto separado por vírgulas	Binário compacto
Tamanho de uma amostra	cerca de 25 bytes	8 bytes
Tempo de linha a 9600	26 ms	8,3 ms
Ler no terminal	Direto	Impossível
Ler em Python	Uma linha de código	Precisa de `struct` e do formato certo
Sobrevive a um byte perdido	Sim — perde-se uma linha	Não — desalinha tudo

CONCEITO

Neste projeto usamos texto. A telemetria envia uma amostra por segundo, e 26 ms a cada 1000 ms ocupam 2,6% do enlace, uma folga de quase quarenta vezes.

O formato binário compensa quando falta banda. Aqui o que pesa mais é o tempo de depuração, e o texto se lê direto no terminal.

```
t_ms,temp_d,alvo_d,aquecedor,comutacoes
125300,398,400,1,17
126300,401,400,0,18
```

13 Previsão para o R10

PREVISÃO

P1. Com $n = 416$, que taxa real você espera medir no osciloscópio? Qual é a largura de um bit?

P2. Você vai abrir o terminal com a taxa configurada em 19200 em vez de 9600. O que espera ver — nada, ou lixo? Justifique com o que a aula disse.

P3. Envie uma linha de telemetria por segundo com a versão bloqueante e ouça o buzzer tocando junto. Descreva o que espera ouvir.

P4. Troque para a versão com fila e interrupção. O que muda no som? E no osciloscópio, no pino TX?

P5. Quantas horas de telemetria a 1 Hz cabem num arquivo de 1 MB? Faça a conta com o formato de texto da aula.

SEM NOTA

Leve esta folha preenchida. Cada pergunta vale 0,4 — 2,0 no total do R10. O arquivo gerado neste roteiro é a matéria-prima do encontro 13 — guarde-o.

14 Exercícios

TAREFA

Exercício 10.1. Um enlace a 9600 funciona entre a placa e o computador, mas falha com um segundo dispositivo, que tem erro de taxa de + 1,8%.

(a) Qual é o erro total do enlace, no pior caso?

(b) Ele cabe no orçamento?

(c) O defeito vai aparecer como tela em branco ou como texto corrompido?

TAREFA

Exercício 10.2. Compare o custo de bloqueio das três coisas que o termostato faz, a 16 MHz, e ordene da pior para a melhor.

(a) Atualizar a tela.

(b) Enviar uma linha de telemetria de 25 bytes a 9600, com a versão bloqueante.

(c) Ler o conversor analógico.

Exercício 10.3. Explique por que `TXIE` precisa ser desligado quando a fila esvazia, e o que acontece se ele ficar sempre ligado. TAREFA

Exercício 10.4. O termostato precisa transmitir uma amostra por segundo durante oito horas para o conjunto de dados do encontro 13. TAREFA

(a) Quantas linhas?

(b) Qual o tamanho do arquivo, no formato de texto da aula?

(c) Qual fração do enlace a 9600 é usada?

(d) Se você quisesse dez amostras por segundo, o formato de texto ainda serviria?

Exercício 10.5. A telemetria funciona, mas a recepção de comandos para de responder depois de alguns minutos. A transmissão continua normal e o programa não trava. TAREFA

(a) Qual indicador você inspecionaria primeiro?

(b) Por que o defeito é permanente, e não intermitente?

(c) Cite duas causas plausíveis, e diga qual delas o encontro 3 já anunciava.

Exercício 10.6. Para cada aplicação, indique o protocolo serial mais provável e a pergunta que decidiu a escolha (com quem, a que distância, quantos dispositivos). TAREFA

(a) Um termômetro digital em cada uma de dez salas de um mesmo corredor, todos lidos pelo mesmo microcontrolador.

(b) Um cartão SD para gravar a telemetria na própria placa.

(c) Doze painéis de controle espalhados por um galpão de 80 m, ligados a um controlador central.

(d) Um relógio de tempo real e uma EEPROM na mesma placa do termostato.

No encontro 11: barramento serial síncrono e memória não volátil. A telemetria guarda o histórico fora da placa, mas o alvo ajustado pelo usuário ainda se perde a cada reset, e o encontro 8 mostrou que resets não são raros neste projeto. NOTA